
APLICACIONES WEB

Edición ampliada Historia, fundamentos, prácticas y buenas prácticas profesionales

Semana 4

JavaScript moderno y programación asíncrona

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Quinto nivel Período Académico 2026–2027 PPA

Autor: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Índice general

4. JavaScript moderno y programación asíncrona	3
Resultado de aprendizaje de la semana	3
4.1. Historia de JavaScript	4
4.1.1. Tabla evolutiva de ECMAScript	4
4.2. El modelo de ejecución: event loop y call stack	5
4.3. Fundamentos modernos del lenguaje	6
4.3.1. <code>let</code> , <code>const</code> y el fin de <code>var</code>	6
4.3.2. Funciones regulares y funciones flecha (<code>=></code>)	7
4.3.3. Template literals y etiquetas	7
4.3.4. Destructuring, spread y rest	8
4.3.5. Optional chaining y nullish coalescing	9
4.3.6. Programación funcional: <code>map</code> , <code>filter</code> y <code>reduce</code>	9
4.4. Cierres (<i>closures</i>)	10
4.5. Prototipos y clases ES6	12
4.6. Manipulación del DOM	13
4.6.1. Selección de elementos	14
4.6.2. Modificación del DOM	14
4.6.3. Eventos y delegación	15
4.7. Programación asíncrona: callbacks, Promises y <code>async/await</code>	17
4.7.1. Callbacks y el <i>callback hell</i>	17
4.7.2. Promises	18
4.7.3. <code>async/await</code> : el azúcar de las Promises	19
4.7.4. Cancelar peticiones con <code>AbortController</code>	20
4.8. Módulos ES (<code>import/export</code>)	22
4.8.1. Importación dinámica	23
4.9. Ejercicios paso a paso en IntelliJ IDEA	23
4.10. Buenas prácticas de JavaScript 2026	27
4.11. Diagrama de arquitectura: cliente con módulos ES	28
Resumen de la semana	28

JavaScript moderno y programación asíncrona

“Cualquier aplicación que pueda escribirse en JavaScript, eventualmente se escribirá en JavaScript.”

— Jeff Atwood, 2007 (*Ley de Atwood*)

Resultado de aprendizaje de la semana

Al concluir la semana 4, el estudiante:

- Comprende la evolución histórica y el modelo de ejecución de JavaScript en el navegador.
- Declara variables con `let/const`, escribe funciones flecha, aplica *destructuring* y *spread*.
- Lee y modifica el árbol DOM, escucha eventos con el patrón `addEventListener` y aplica delegación de eventos.
- Comprende el *event loop*, la pila de llamadas y la cola de tareas.
- Escribe funciones de orden superior con `.map()`, `.filter()` y `.reduce()`.
- Modela y reutiliza comportamiento con cierres (*closures*) y con clases ES6.
- Consume APIs REST con `fetch` y `async/await`, encadena Promises y gestiona errores con `try/catch`.
- Organiza el código en módulos ES con `import/export`.
- Aplica las doce buenas prácticas profesionales de JavaScript 2026 [?, ?].

4.1 Historia de JavaScript

JavaScript nació en 1995 como un lenguaje íde juguetez creado en diez días. Hoy es la *lingua franca* del navegador, ejecuta servidores enteros (Node.js), corre en satélites y dispositivos IoT, y es el lenguaje más usado del mundo según el índice TIOBE 2025. Conocer su historia ayuda a entender por qué tiene particularidades sintácticas y por qué evoluciona tan rápido.

De diez días de diseño a la *lingua franca* de la web

En mayo de 1995, Netscape contrató a **Brendan Eich** para crear un lenguaje de scripting que se ejecutara en el navegador. Le dieron *diez días*. El resultado, llamado originalmente **Mocha**, luego **LiveScript** y finalmente **JavaScript** (1996, por razones de marketing vinculadas a la popularidad de Java), reunía rasgos memorables: tipado dinámico, *first-class functions*, prototipos en lugar de clases, `undefined` y `null` como dos formas distintas de *ínadaž* [?].

Microsoft respondió con JScript en Internet Explorer 3 (1996), provocando una guerra de incompatibilidades que se cerró con la estandarización ECMA en 1997 (**ECMAScript 1**). Durante una década, JavaScript fue considerado un lenguaje de segunda clase. El cambio llegó con el artículo de Jesse James Garrett que acuñó **Ajax** (2005) [?] y con el motor V8 de **Google Chrome** (2008), que compilaba JS a código nativo y era hasta 100E más rápido que sus predecesores. Ryan Dahl aprovechó V8 para crear **Node.js** (2009), llevando JS al servidor.

ECMAScript 2015 (ES6) fue el mayor salto del lenguaje: clases, módulos, `let/const`, funciones flecha, Promises y *destructuring*. Desde entonces el estándar se actualiza cada año. ECMAScript 2024 (ES15) es hoy el estándar vigente [?]; ES2026 incorpora `Array.fromAsync`, `Promise.try` y *records/tuples* inmutables.

4.1.1 Tabla evolutiva de ECMAScript

Tabla 4.1: Versiones más relevantes de ECMAScript.

Versión	Año	Aportes principales
ES3	1999	Expresiones regulares, <code>try/catch</code> , manejo de excepciones.
ES5	2009	<code>strict mode</code> , JSON nativo, <code>Array.forEach</code> , getters/setters.
ES6 (ES2015)	2015	<code>let/const</code> , clases, módulos, arrow functions, Promises, template literals, <i>destructuring</i> , <i>spread</i> , <code>Map/Set</code> .
ES2016	2016	<code>Array.includes()</code> , operador exponente <code>**</code> .
ES2017	2017	<code>async/await</code> , <code>Object.entries()</code> , <code>String.padStart()</code> .
ES2018	2018	<i>Rest/spread</i> en objetos, <code>Promise.finally()</code> , iteradores asíncronos.
ES2019	2019	<code>Array.flat()</code> , <code>Object.fromEntries()</code> , <code>String.trimStart()</code> .
ES2020	2020	Optional chaining (<code>?.</code>), nullish coalescing (<code>??</code>), <code>BigInt</code> .
ES2021	2021	<code>String.replaceAll()</code> , <code>Promise.any()</code> , <code>WeakRef</code> .
ES2022	2022	Top-level <code>await</code> , campos privados (<code>#</code>), <code>Array.at()</code> .
ES2023	2023	<code>Array.findLast()</code> , <code>Array.toSorted()</code> , <code>Array.toReversed()</code> .
ES2024	2024	<code>Object.groupBy()</code> , <code>ArrayBuffer</code> transferible, <code>Promise.withResolvers()</code> .
ES2026 [?]	2026	<code>Array.fromAsync()</code> , <code>Promise.try()</code> , <i>records</i> y <i>tuples</i> inmutables.

4.2 El modelo de ejecución: event loop y call stack

Antes de escribir código asíncrono es indispensable comprender *por qué* JavaScript necesita asincronía. El lenguaje corre en un único hilo (*single-threaded*): solo ejecuta una instrucción a la vez. Si una operación lenta una petición HTTP, una lectura de disco bloqueara el hilo, la página quedaría congelada. El **event loop** resuelve este problema delegando las operaciones lentas al entorno (navegador o Node.js) y procesando sus resultados cuando el hilo principal está libre.

Glosario del modelo de ejecución

Call Stack

Pila de llamadas donde se apilan los marcos de ejecución de las funciones activas. LIFO (último en entrar, primero en salir).

Web APIs

Servicios del navegador que gestionan operaciones asíncronas: `setTimeout`, `fetch`, `XMLHttpRequest`, eventos del DOM, etc.

Callback / Task Queue

Cola donde las Web APIs depositan los callbacks listos para ejecutarse.

Microtask Queue

Cola de mayor prioridad para callbacks de Promises (`.then()`, `await`) y `MutationObserver`.

Event Loop

Bucle que comprueba continuamente si la Call Stack está vacía; si lo está, mueve primero todas las microtareas y luego una tarea de la Task Queue.

La Figura 4.1 muestra el modelo completo.

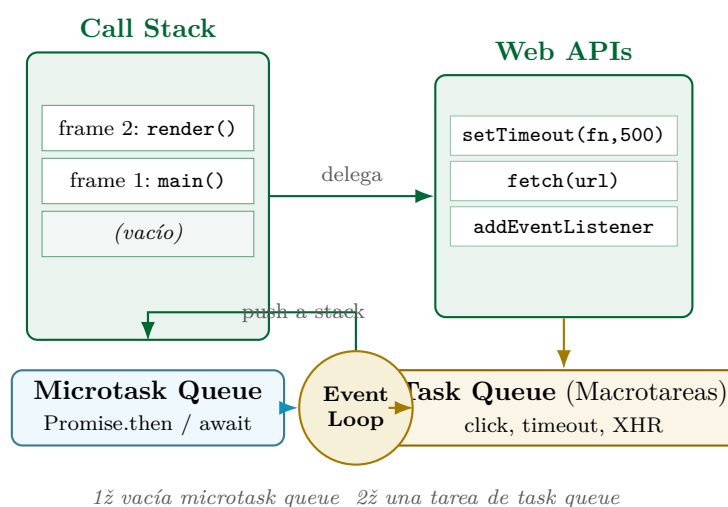


Figura 4.1: Modelo de ejecución de JavaScript: call stack, Web APIs, microtask queue, task queue y event loop.

Ejemplo 4.0 Orden de ejecución del event loop

El siguiente código ilustra la prioridad de las colas.

```
console.log("1 - sincrono");           // stack
    inmediato

setTimeout(() => console.log("4 - macrotarea"), 0); // task queue

Promise.resolve()
    .then(() => console.log("3 - microtarea"));      // microtask
    queue

console.log("2 - sincrono");           // stack
    inmediato
```

Listing 4.1: Orden observable en la consola

Salida en consola:

```
1 - sincrono
2 - sincrono
3 - microtarea
4 - macrotarea
```

Explicación: Las dos líneas síncronas se ejecutan primero (stack). Luego se vacía la microtask queue (la Promise). Solo entonces el event loop procesa la macrotarea del `setTimeout`, aunque su retardo sea cero.

4.3 Fundamentos modernos del lenguaje

Esta sección cubre las características del lenguaje que todo desarrollador JavaScript 2026 debe dominar antes de abordar el DOM o el código asíncrono.

4.3.1 `let`, `const` y el fin de `var`

La palabra clave `var` tenía dos defectos graves: *hoisting* (las variables se elevaban silenciosamente al inicio del *scope*) y *function scope* en lugar de *block scope*. ES6 los corrigió con `let` (mutable, block-scoped) y `const` (inmutable en referencia, block-scoped). Véase el Listado 4.2.

```
// REGLA PRACTICA: const por defecto, let si muta, var NUNCA
const PI = 3.14159;           // inmutable; error si se reasigna
let contador = 0;             // mutable, scope de bloque
var legacy = "evitar";        // hoisted, scope de funcion - NO usar

// Hoisting de var (trampa clasica):
console.log(legacy2);          // undefined, NO un ReferenceError
var legacy2 = "peligroso";

// Block scope de let/const:
{
    let local = 10;
```

```
const MAX = 100;
}
// console.log(local); // ReferenceError: local is not defined
```

Listing 4.2: Variables en JavaScript moderno

4.3.2 Funciones regulares y funciones flecha (=>)

Las funciones flecha son más concisas y, crucialmente, **no enlazan su propio this**: heredan el **this** del contexto léxico donde fueron definidas. Esto las hace ideales como callbacks pero inadecuadas como métodos de objetos cuando se necesita acceso a la instancia. Véase el Listado 4.3.

```
1 // Funcion regular con declaracion
2 function sumar(a, b) {
3     return a + b;
4 }
5
6 // Funcion flecha equivalente (cuerpo conciso)
7 const sumar = (a, b) => a + b;
8
9 // Arrow sin parametros
10 const ahora = () => new Date().toISOString();
11
12 // Arrow con cuerpo completo
13 const dividir = (a, b) => {
14     if (b === 0) throw new Error("Division por cero");
15     return a / b;
16 };
17
18 // Diferencia critica: this lexico
19 function Temporizador() {
20     this.segundos = 0;
21     // INCORRECTO con funcion regular: this sera undefined en strict
22     // setInterval(function() { this.segundos++; }, 1000);
23
24     // CORRECTO con arrow: hereda this del constructor
25     setInterval(() => {
26         this.segundos++;
27         console.log(this.segundos);
28     }, 1000);
29 }
30 const t = new Temporizador();
```

Listing 4.3: Funciones regulares vs. funciones flecha

4.3.3 Template literals y etiquetas

Los *template literals* (backticks) permiten interpolación de expresiones, cadenas multilínea y *tagged templates* para sanitización o internacionalización. Véase el Listado 4.4.

```
const nombre = "Maria";
const edad   = 22;
```

```

// Concatenacion clasica (evitar)
const msg1 = "Hola " + nombre + ", tienes " + edad + " años.";

// Template literal (preferido)
const msg2 = `Hola ${nombre}, tienes ${edad} años.`;

// Expresion compleja dentro de ${}
const precio = 9.5;
const iva = 0.15;
console.log(`Total con IVA: ${((precio * (1 + iva)).toFixed(2))}`);

// Cadena multilinea
const html = `
<div class="tarjeta">
<h2>${nombre}</h2>
<p>Edad: ${edad}</p>
</div>
`;

```

Listing 4.4: Template literals

4.3.4 Destructuring, spread y rest

Destructuring extrae valores de objetos y arrays con sintaxis declarativa concisa. *Spread* (...) expande colecciones en distintos contextos; *rest* recoge el resto de elementos en un parámetro. Véase el Listado 4.5.

```

1 // --- Destructuring de objeto ---
2 const usuario = { nombre: "Ana", edad: 22, rol: "admin" };
3 const { nombre, rol } = usuario;           // extrae dos campos
4 const { edad: anios } = usuario;          // renombra: anios = 22
5 const { pais = "EC" } = usuario;          // valor por defecto
6
7 // --- Destructuring de array ---
8 const colores = ["rojo", "verde", "azul"];
9 const [primero, , tercero] = colores;     // omite el segundo
10
11 // --- Rest en parametros ---
12 function suma(...numeros) {
13     return numeros.reduce((acc, n) => acc + n, 0);
14 }
15 console.log(suma(1, 2, 3, 4));             // 10
16
17 // --- Spread para copiar/combinar ---
18 const base = { x: 1, y: 2 };
19 const copia = { ...base };                 // copia superficial
20 const amplia = { ...base, z: 3 };          // extiende
21 const nums = [1, 2, 3];
22 const mas = [...nums, 4, 5];               // concatena
23
24 // --- Destructuring en parametros ---

```



```
25 function mostrar({ nombre, edad = 0 }) {  
26   console.log(`${nombre} tiene ${edad} años`);  
27 }  
28 mostrar(usuario);
```

Listing 4.5: Destructuring, spread y rest

4.3.5 Optional chaining y nullish coalescing

ES2020 introdujo dos operadores que simplifican el acceso seguro a propiedades que pueden no existir y el manejo de valores nulos o indefinidos. Véase el Listado 4.6.

```
const resp = {  
  data: {  
    usuario: { nombre: "Luis", avatar: null }  
  }  
};  
  
// Sin optional chaining (propenso a TypeError)  
// const nombre = resp.data.usuario.nombre; // ok  
// const ciudad = resp.data.usuario.ciudad.nombre; // TypeError!  
  
// Con optional chaining  
const nombre = resp?.data?.usuario?.nombre; // "Luis"  
const ciudad = resp?.data?.usuario?.ciudad?.nombre; // undefined  
  
// Invocar metodo opcional  
const longitud = resp?.data?.usuario?.nombre?.length; // 4  
  
// Nullish coalescing: ?? retorna el lado derecho solo si  
// el izquierdo es null o undefined (NO si es 0 o "")  
const avatar = resp.data.usuario.avatar ?? "/img/default.png";  
const total = 0 ?? 100; // 0 (0 no es null ni undefined)  
const total2 = null ?? 100; // 100
```

Listing 4.6: Optional chaining (?.) y nullish coalescing (??)

4.3.6 Programación funcional: map, filter y reduce

Los tres métodos de array más usados en JavaScript moderno implementan el paradigma funcional: transformar, filtrar y reducir datos sin mutar el array original. Véase el Listado 4.7.

```
1 const productos = [  
2   { nombre: "Camiseta", precio: 15, activo: true },  
3   { nombre: "Pantalon", precio: 45, activo: true },  
4   { nombre: "Sombrero", precio: 8, activo: false },  
5   { nombre: "Zapatos", precio: 60, activo: true },  
6 ];  
7  
8 // MAP: transforma cada elemento -> nuevo array  
9 const nombres = productos.map(p => p.nombre);  
10 // ["Camiseta", "Pantalon", "Sombrero", "Zapatos"]
```

```
11
12  const conIva = productos.map(p => ({
13    ...p,
14    precioFinal: +(p.precio * 1.15).toFixed(2)
15  }));
16
17  // FILTER: conserva solo los que cumplen la condicion
18  const activos = productos.filter(p => p.activo);
19  // 3 productos (sin Sombrero)
20
21  // REDUCE: acumula un unico valor
22  const totalActivos = productos
23    .filter(p => p.activo)
24    .reduce((suma, p) => suma + p.precio, 0);
25  // 120
26
27  // CADENA fluida: nombres de activos con precio < 50
28  const baratos = productos
29    .filter(p => p.activo && p.precio < 50)
30    .map(p => p.nombre);
31  // ["Camiseta", "Pantalon"]
```

Listing 4.7: Funciones de orden superior sobre arrays

4.4 Cierres (*closures*)

Un **closure** es una función que recuerda el entorno léxico en el que fue creada, incluso después de que dicho entorno haya terminado de ejecutarse. Es uno de los conceptos más poderosos y más incomprensidos de JavaScript.

Definición formal de closure

Una **closure** es la combinación de una función y el entorno léxico en el que fue declarada. Ese entorno consiste en las variables locales que estaban en *scope* en el momento de la creación de la función [?].

Ejemplo 4.1 (RESUELTO) Contador con closure

Objetivo: comprender cómo un closure mantiene estado privado sin variables globales.

```
1  "use strict";
2
3  // La funcion crearContador devuelve un objeto
4  // cuyas funciones comparten la variable privada 'cuenta'
5  function crearContador(inicio = 0) {
6    let cuenta = inicio;           // variable privada (no
7                                   accesible desde fuera)
8
9    return {
10      incrementar() { cuenta++;    return cuenta; },
```

```

10     decrementar() { cuenta--;      return cuenta; },
11     reiniciar()   { cuenta = inicio; return cuenta; },
12     valor()       { return cuenta; }
13 };
14 }
15
16 const c1 = crearContador(10);
17 const c2 = crearContador();    // cada instancia tiene su
    propio 'cuenta'
18
19 console.log(c1.incrementar()); // 11
20 console.log(c1.incrementar()); // 12
21 console.log(c2.valor());        // 0 (independiente de c1)
22 console.log(c1.reiniciar());    // 10

```

Listing 4.8: Contador encapsulado con closure

Verificación: intentar acceder a `c1.cuenta` devuelve `undefined`: la variable es verdaderamente privada. No existe ninguna API del navegador para acceder al cierre desde el exterior.

Ejemplo 4.2 (RESUELTO) Memoización con closure

Objetivo: cachear resultados de funciones costosas.

```

1      "use strict";
2
3      function memoizar(fn) {
4          const cache = new Map();                // almacen
          privado
5
6          return function (...args) {
7              const clave = JSON.stringify(args);    // clave
              unica por argumentos
8              if (cache.has(clave)) {
9                  console.log("HIT cache:", clave);
10                 return cache.get(clave);
11             }
12             const resultado = fn.apply(this, args);
13             cache.set(clave, resultado);
14             return resultado;
15         };
16     }
17
18     // Funcion costosa de ejemplo
19     function fibonacci(n) {
20         if (n <= 1) return n;
21         return fibonacci(n - 1) + fibonacci(n - 2);
22     }
23
24     const fibMemo = memoizar(fibonacci);

```

```

25 console.log(fibMemo(35)); // calcula (tarda ~ms)
26 console.log(fibMemo(35)); // HIT cache: instantaneo

```

Listing 4.9: Memoización genérica

4.5 Prototipos y clases ES6

JavaScript usa herencia basada en *prototipos*, no en clases clásicas. La sintaxis `class` de ES6 es *azúcar sintáctica* sobre el sistema de prototipos: lo hace más legible sin cambiar el modelo subyacente.

El malentendido de las *ñclasesz* en JavaScript

Cuando Java y C++ dominaban la programación orientada a objetos, Brendan Eich eligió un modelo diferente inspirado en **Self** (Xerox PARC, 1986): los objetos heredan directamente de otros objetos a través de una cadena de prototipos, sin necesidad de clases. Esta decisión fue pragmática diez días no dan para más pero resultó enormemente poderosa.

La comunidad pidió clases durante una década. ES6 (2015) las introdujo, pero como *syntactic sugar*: internamente siguen siendo funciones constructoras y prototipos. El desarrollador que comprende esto entiende por qué `typeof MiClase` es `"function"` y por qué la cadena de prototipos sigue siendo accesible mediante `Object.getPrototypeOf()`.

```

1  "use strict";
2
3  // Clase base con campo privado (ES2022)
4  class Figura {
5      #color;                // campo privado
6
7      constructor(color = "negro") {
8          this.#color = color;
9      }
10
11     get color() { return this.#color; } // getter publico
12
13     describir() {
14         return `Figura de color ${this.#color}`;
15     }
16
17     // Metodo estatico: pertenece a la clase, no a la instancia
18     static esValido(obj) {
19         return obj instanceof Figura;
20     }
21 }
22
23 // Herencia con extends y super
24 class Rectangulo extends Figura {
25     #ancho;

```

```

26     #alto;
27
28     constructor(ancho, alto, color) {
29         super(color);           // llama al constructor padre
30         this.#ancho = ancho;
31         this.#alto = alto;
32     }
33
34     get area() { return this.#ancho * this.#alto; }
35     get perimetro() { return 2 * (this.#ancho + this.#alto); }
36
37     describir() {                // sobrescribe (override)
38         return `${super.describir()}, ${this.#ancho}x${this.#alto}cm`;
39     }
40 }
41
42 const r = new Rectangulo(10, 5, "azul");
43 console.log(r.describir());    // Figura de color azul, 10x5cm
44 console.log(r.area);           // 50
45 console.log(Figura.esValido(r)); // true

```

Listing 4.10: Clases ES6 con herencia y campos privados

Prototype vs. Class: ¿qué pasa bajo el capó?

```

// Con clase ES6
class Animal { hablar() { return "..."; } }

// Es equivalente a (pre-ES6):
function Animal() {}
Animal.prototype.hablar = function() { return "..."; };

// Verificacion
console.log(typeof Animal);           // "function"
console.log(Animal.prototype.hablar); // [Function: hablar]

```

4.6 Manipulación del DOM

El **DOM** (*Document Object Model*) es la representación en memoria de la página HTML, expuesta como un árbol de nodos que JavaScript puede leer y modificar. Cada elemento HTML es un objeto con propiedades y métodos.

El árbol del DOM

El DOM tiene una raíz (`document`), que contiene `document.documentElement` (`<html>`), y de ahí descienden `<head>` y `<body>`. Cada nodo puede ser de tipo `Element`, `Text` o `Comment`. La manipulación del DOM debe ser mínima y eficiente: cada modificación puede provocar *reflow* y *repaint* en el navegador.

4.6.1 Selección de elementos

```
// Selección por ID (devuelve un elemento o null)
const titulo = document.getElementById("titulo");

// Selección con selector CSS (primero que coincide)
const btn = document.querySelector(".btn-primario");

// Selección múltiple (devuelve NodeList estática)
const items = document.querySelectorAll("ul li");

// Iteración sobre NodeList
items.forEach(item => console.log(item.textContent));
// Con spread a Array para usar .map()
const textos = [...items].map(li => li.textContent);

// Relaciones entre nodos
const padre = btn.parentElement;
const hijos = padre.children;           // HTMLCollection
const sig = btn.nextElementSibling;
```

Listing 4.11: Métodos de selección del DOM

4.6.2 Modificación del DOM

```
// Crear nodo
const parrafo = document.createElement("p");
parrafo.textContent = "Hola, DOM!";           // texto plano (seguro vs XSS)
parrafo.className = "resaltado";
parrafo.dataset.id = "42";                    // <p data-id="42">

// Agregar al documento
document.body.appendChild(parrafo);           // al final
document.body.prepend(parrafo);               // al inicio
// insertBefore(nuevo, referencia)

// Modificar atributos
parrafo.setAttribute("lang", "es");
parrafo.removeAttribute("lang");

// Modificar estilos en línea (preferir clases CSS)
parrafo.style.color = "red";
parrafo.style.fontWeight = "bold";

// Agregar/quitar clases
parrafo.classList.add("activo");
parrafo.classList.remove("resaltado");
parrafo.classList.toggle("visible");          // alterna

// Eliminar nodo
parrafo.remove();
```

Listing 4.12: Crear, modificar y eliminar nodos

4.6.3 Eventos y delegación

```

1      "use strict";
2
3      // Patron basico
4      const boton = document.querySelector("#enviar");
5      boton.addEventListener("click", (e) => {
6          e.preventDefault();                // cancela accion por defecto
7          console.log("clic en", e.target.id);
8      });
9
10     // Delegacion de eventos: un solo listener en el contenedor
11     // en lugar de uno por cada hijo (eficiente para listas dinamicas)
12     const lista = document.querySelector("#lista");
13     lista.addEventListener("click", (e) => {
14         const item = e.target.closest("li");    // busca el li mas cercano
15         if (!item) return;                    // clic fuera de un item
16         item.classList.toggle("completado");
17         console.log("toggled:", item.dataset.id);
18     });
19
20     // Eventos de teclado
21     document.addEventListener("keydown", (e) => {
22         if (e.key === "Escape") cerrarModal();
23         if (e.ctrlKey && e.key === "s") guardar(e);
24     });
25
26     // Eliminar listener (se necesita referencia a la funcion)
27     function manejarScroll() { /* ... */ }
28     window.addEventListener("scroll", manejarScroll);
29     window.removeEventListener("scroll", manejarScroll);

```

Listing 4.13: Eventos con addEventListener y delegación

Ejemplo 4.3 (RESUELTO) Lista de tareas dinámica

Objetivo: aplicar DOM, eventos, delegación y validación de entrada en una mini-aplicación completa.

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
    initial-scale=1">
<title>Mis tareas</title>
<style>
body      { font-family: sans-serif; max-width: 600px; margin:
    2rem auto; }
ul        { list-style: none; padding: 0; }
li        { display: flex; justify-content: space-between;
    padding: .5rem; border-bottom: 1px solid #eee; }
li.hecha  { text-decoration: line-through; color: #999; }

```

```
button    { cursor: pointer; }
</style>
</head>
<body>
<h1>Mis tareas</h1>
<form id="form-tarea">
<input id="texto-tarea" type="text" placeholder="Nueva tarea..."
minlength="3" required>
<button type="submit">Agregar</button>
</form>
<ul id="lista"></ul>
<script src="tareas.js" type="module"></script>
</body>
</html>
```

Listing 4.14: tareas.html

```
1      "use strict";
2
3      const form = document.getElementById("form-tarea");
4      const input = document.getElementById("texto-tarea");
5      const lista = document.getElementById("lista");
6      let idAuto = 0;
7
8      // Agregar tarea
9      form.addEventListener("submit", (e) => {
10         e.preventDefault();
11         const texto = input.value.trim();
12         if (texto.length < 3) {
13             input.setCustomValidity("Minimo 3 caracteres");
14             input.reportValidity();
15             return;
16         }
17         input.setCustomValidity("");
18         agregarTarea(texto);
19         input.value = "";
20         input.focus();
21     });
22
23     function agregarTarea(texto) {
24         const li = document.createElement("li");
25         li.dataset.id = ++idAuto;
26         li.textContent = texto;
27
28         const btn = document.createElement("button");
29         btn.textContent = "X";
30         btn.setAttribute("aria-label", "Eliminar tarea");
31         li.appendChild(btn);
32         lista.appendChild(li);
33     }
34
```



```

35 // Delegacion: clic en lista para marcar/eliminar
36 lista.addEventListener("click", (e) => {
37     const li = e.target.closest("li");
38     if (!li) return;
39
40     if (e.target.tagName === "BUTTON") {
41         li.remove(); // eliminar
42     } else {
43         li.classList.toggle("hecha"); // marcar
44         // completa
45     }
46 });

```

Listing 4.15: tareas.js

Resultado esperado:

Mis tareas

Estudiar JavaScript

X

[completada] Hacer ejercicios HTML

X

Practicar Flexbox

X

La segunda tarea está marcada como completada (tachada) al hacer clic sobre su texto. El botón **X** la elimina.

4.7 Programación asíncrona: callbacks, Promises y async/await

La asincronía es ineludible en el navegador: peticiones HTTP, lectura de archivos, temporizadores y eventos del usuario no deben bloquear la interfaz. JavaScript evolucionó a través de tres mecanismos sucesivos para manejarla.

4.7.1 Callbacks y el *callback hell*

El patrón más antiguo usa funciones de retorno (*callbacks*) que se pasan como argumento. El problema surge cuando se encadenan: se forma una pirámide de código difícil de leer y mantener, conocida como *callback hell* o *pyramid of doom*. Véase el Listado 4.16.

```

// ANTI-PATRON: callback hell
iniciarSesion(usuario, pass, (err, token) => {
    if (err) return manejarError(err);
    obtenerPerfil(token, (err, perfil) => {
        if (err) return manejarError(err);
        cargarPreferencias(perfil.id, (err, prefs) => {
            if (err) return manejarError(err);
            renderizar(perfil, prefs); // profundidad maxima: ilegible
        });
    });
});

```

```
});
});
```

Listing 4.16: *Callback hell* (patrón a evitar)

4.7.2 Promises

Una **Promise** representa el resultado eventual de una operación asíncrona. Puede estar en tres estados: **pending**, **fulfilled** o **rejected**. Permite encadenar operaciones con `.then()` y capturar errores con `.catch()`, eliminando la pirámide. Véase el Listado 4.17.

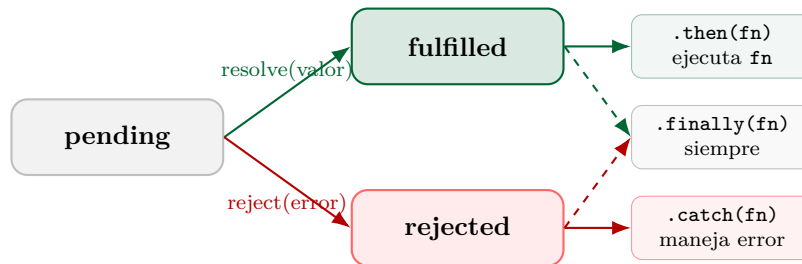


Figura 4.2: Estados y transiciones de una Promise.

```

1  // Crear una Promise manualmente
2  function esperar(ms) {
3      return new Promise(resolve => setTimeout(resolve, ms));
4  }
5
6  // Encadenar con .then() / .catch() / .finally()
7  esperar(500)
8  .then(() => {
9      console.log("Paso 1 completado");
10     return esperar(300); // devuelve otra Promise
11 })
12 .then(() => {
13     console.log("Paso 2 completado");
14     throw new Error("Fallo simulado");
15 })
16 .catch(err => {
17     console.error("Error capturado:", err.message);
18 })
19 .finally(() => {
20     console.log("Siempre se ejecuta");
21 });
22
23 // Promise.all: espera que TODAS se resuelvan
24 const [datos, config] = await Promise.all([
25     fetch("/api/datos").then(r => r.json()),
26     fetch("/api/config").then(r => r.json())
27 ]);
28
29 // Promise.allSettled: espera TODAS sin importar resultado
30 const resultados = await Promise.allSettled([
31     fetch("/api/a").then(r => r.json()),

```

```

32     fetch("/api/b").then(r => r.json())
33   ];
34   resultados.forEach(r => {
35     if (r.status === "fulfilled") console.log("OK:", r.value);
36     else console.error("Fallo:", r.reason.message);
37   });
38
39   // Promise.race: la primera que resuelva (o rechace) gana
40   const timeout = new Promise( (_, reject) =>
41     setTimeout(() => reject(new Error("Timeout")), 3000)
42   );
43   const dato = await Promise.race([fetch("/api/lento"), timeout]);

```

Listing 4.17: Crear y encadenar Promises

4.7.3 async/await: el azúcar de las Promises

`async/await` es sintaxis que convierte el código asíncrono en algo que parece síncrono, eliminando el encadenamiento de `.then()`. Una función `async` *siempre* devuelve una Promise. `await` suspende la ejecución de la función hasta que la Promise se resuelve o rechaza. Véase el Listado 4.18.

```

1     "use strict";
2
3     // Funcion auxiliar: verifica la respuesta HTTP
4     async function respuestaSegura(res) {
5       if (!res.ok) {
6         const cuerpo = await res.text().catch(() => "");
7         throw new Error(`HTTP ${res.status}: ${cuerpo}`);
8       }
9       return res.json();
10    }
11
12    // Funcion principal async
13    async function obtenerUsuario(id) {
14      try {
15        const res = await
16          fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
17        const usuario = await respuestaSegura(res);
18        return usuario;
19      } catch (err) {
20        console.error("Error al obtener usuario:", err.message);
21        return null; // valor centinela
22      }
23    }
24
25    // Llamada al nivel de modulo (top-level await en ES2022)
26    const usuario = await obtenerUsuario(1);
27    if (usuario) {
28      console.log(`${usuario.name} <${usuario.email}>`);
29    }

```

Listing 4.18: `async/await` con manejo de errores

4.7.4 Cancelar peticiones con `AbortController`

```
"use strict";

let controlador = null;

async function buscar(termino) {
  // Cancelar búsqueda anterior si aun esta en curso
  if (controlador) controlador.abort();
  controlador = new AbortController();
  const { signal } = controlador;

  try {
    const res = await fetch(
      `https://api.ejemplo.com/buscar?q=${encodeURIComponent(termino)}`,
      { signal }
    );
    const data = await res.json();
    renderizarResultados(data);
  } catch (err) {
    if (err.name === "AbortError") {
      console.log("Búsqueda cancelada (nueva consulta)");
    } else {
      console.error("Error de red:", err.message);
    }
  }
}

// Uso: buscar en tiempo real al escribir (debounced)
document.querySelector("#buscador")
  .addEventListener("input", e => buscar(e.target.value));
```

Listing 4.19: Cancelar fetch con `AbortController`

Ejemplo 4.4 (RESUELTO) Panel de clima con fetch

Objetivo: consumir una API REST pública, mostrar datos en el DOM y manejar errores de red y de lógica de aplicación.

```
1  "use strict";
2
3  const API_KEY = "DEMO_KEY"; // reemplazar con
   clave real
4  const URL_BASE = "https://api.open-meteo.com/v1/forecast";
5
6  // Datos de ciudades (sustituye llamada a geocoding por brevedad)
7  const CIUDADES = {
8    quevedo: { lat: -1.02, lon: -79.46 },
9    guayaquil: { lat: -2.18, lon: -79.89 },
```

```
10     quito:    { lat: -0.22, lon: -78.51 }
11   };
12
13   async function obtenerClima(ciudad) {
14     const coords = CIUDADES[ciudad.toLowerCase()];
15     if (!coords) throw new Error('Ciudad desconocida: ${ciudad}');
16
17     const params = new URLSearchParams({
18       latitude: coords.lat,
19       longitude: coords.lon,
20       current:  "temperature_2m,weathercode,wind_speed_10m",
21       timezone: "America/Guayaquil"
22     });
23
24     const res = await fetch(`${URL_BASE}?${params}`);
25     if (!res.ok) throw new Error('HTTP ${res.status}');
26     return res.json();
27   }
28
29   function renderClima(data, ciudad) {
30     const { temperature_2m, wind_speed_10m } = data.current;
31     document.getElementById("ciudad").textContent = ciudad;
32     document.getElementById("temp").textContent =
33       `${temperature_2m} degC`;
34     document.getElementById("viento").textContent =
35       `${wind_speed_10m} km/h`;
36   }
37
38   // Manejador del boton
39   document.getElementById("btn-clima").addEventListener("click",
40     async () => {
41       const ciudad =
42         document.getElementById("input-ciudad").value.trim();
43       const estado = document.getElementById("estado");
44
45       if (!ciudad) { estado.textContent = "Ingrese una ciudad.";
46         return; }
47
48       estado.textContent = "Cargando...";
49       try {
50         const data = await obtenerClima(ciudad);
51         renderClima(data, ciudad);
52         estado.textContent = "";
```

Listing 4.20: clima.js – modulo completo

Cómo probarlo: crear clima.html con un `<input id="input-ciudad">`, `<button`

`id="btn-clima">, <span id="ciudad">, <span id="temp">, <span id="viento"> y <span id="estado">. Cargar en el navegador y escribir quevedo.`

4.8 Módulos ES (import/export)

Los módulos ES (ESM) son la solución nativa del lenguaje para organizar el código en ficheros independientes con un espacio de nombres propio. Cada módulo tiene su propio scope: no hay variables globales implícitas.

Del global soup a los módulos ES

Durante más de quince años, el único mecanismo de modularización en JavaScript era concatenar scripts en el HTML o usar patrones como el *Revealing Module Pattern* (2007, Christian Heilmann) basado en closures. La comunidad inventó soluciones como **CommonJS** (Node.js, 2009, con `require()`) y **AMD** (RequireJS, 2010, con `define()`). La guerra de formatos terminó con ES2015: los **módulos ES nativos** son el estándar oficial. Hoy todos los navegadores modernos y Node.js 12+ los soportan sin transpilación.

```

1 // ===== archivo: matematica.js =====
2 // Exportacion nombrada
3 export const PI = 3.14159265;
4
5 export function circunferencia(r) {
6   return 2 * PI * r;
7 }
8
9 export function area(r) {
10   return PI * r ** 2;
11 }
12
13 // Exportacion por defecto (solo una por modulo)
14 export default class Calculadora {
15   sumar(a, b) { return a + b; }
16   restar(a, b) { return a - b; }
17 }
18
19 // ===== archivo: main.js =====
20 // Importacion nombrada
21 import { PI, circunferencia, area } from "./matematica.js";
22
23 // Importacion por defecto
24 import Calculadora from "./matematica.js";
25
26 // Importacion con alias
27 import { circunferencia as circ } from "./matematica.js";
28
29 // Importacion de todo el modulo como objeto
30 import * as Mat from "./matematica.js";
31
```

```

32 console.log(PI); // 3.14159265
33 console.log(circunferencia(5)); // 31.416...
34 const calc = new Calculadora();
35 console.log(calc.sumar(3, 4)); // 7
36 console.log(Mat.area(3)); // 28.274...

```

Listing 4.21: Módulos ES: export e import

```

<!-- type="module" activa el scope de modulo -->
<!-- y habilita top-level await -->
<script type="module" src="main.js"></script>
<!-- El atributo defer es implicito en type="module" -->

```

Listing 4.22: Usar módulos en HTML

CORS y módulos locales

Los módulos ES **no funcionan** cuando se abre el archivo HTML directamente con `file://`: el navegador bloquea la carga por política de origen cruzado (CORS). Se necesita un servidor HTTP. En IntelliJ IDEA basta con hacer clic derecho sobre el `.html` y elegir **Open in Browser** (IntelliJ inicia automáticamente un servidor local).

4.8.1 Importación dinámica

La importación dinámica (`import()`) carga un módulo bajo demanda. Es clave para el *code splitting* y la carga diferida (*lazy loading*). Véase el Listado 4.23.

```

document.getElementById("btn-mapa")
  .addEventListener("click", async () => {
    // El modulo solo se carga cuando el usuario hace clic
    const { iniciarMapa } = await import("./mapa.js");
    iniciarMapa("#contenedor-mapa");
  });

```

Listing 4.23: Importación dinámica bajo demanda

4.9 Ejercicios paso a paso en IntelliJ IDEA

Los siguientes ejercicios incluyen instrucciones completas para reproducirlos en IntelliJ IDEA Ultimate con la licencia educativa gratuita de JetBrains.

Ejercicio 4.1 (PROPUESTO) — Calculadora interactiva

Enunciado. Implementar una calculadora web con JavaScript puro que soporte las cuatro operaciones básicas y muestre el resultado en tiempo real.

Criterios de aceptación:

- Usa `const/let` y funciones flecha.
- Valida división por cero con mensaje amigable.
- Tiene `"use strict"` y `type="module"`.

- El historial de operaciones se muestra con DOM.
- Limpia el historial con un botón.

Pistas: función `calcular(a, op, b)` devuelve el resultado o lanza un `Error`; un listener de `submit` en el formulario llama a esa función y actualiza el DOM.

Ejercicio 4.2 (PROPUESTO) — Buscador de Pokémon

Enunciado. Construir una página que consuma la PokeAPI pública (<https://pokeapi.co/api/v2/pokemon/{nombre}>) con `fetch` y `async/await`, mostrando imagen, tipos y estadísticas del Pokémon buscado.

Criterios de aceptación:

- Maneja errores HTTP 404 con mensaje al usuario.
- Usa `AbortController` para cancelar búsquedas anteriores.
- Muestra un indicador de *¡cargando...* mientras espera la respuesta.
- El código está organizado en al menos dos módulos ES.

Ejercicio 4.3 (RESUELTO) — Galería de usuarios con módulos

Enunciado. Crear una galería de usuarios consumiendo la API pública <https://randomuser.me/api/?results=12>. Separar la lógica en tres módulos: `api.js`, `ui.js` y `main.js`.

Pasos en IntelliJ IDEA Ultimate:

Paso 1 Crear el proyecto.

En IntelliJ, abrir el menú **File** → **New** → **Project...**, seleccionar **Empty Project**, escribir el nombre `galeria-usuarios` y confirmar con **Create**. *Por qué:* un proyecto vacío evita plantillas innecesarias para un ejercicio de JavaScript puro.

Paso 2 Crear la estructura de carpetas.

En el panel **Project**, hacer clic derecho sobre la raíz y elegir **New** → **Directory**. Crear la carpeta `src`. Dentro de ella crear tres archivos JavaScript: `api.js`, `ui.js`, `main.js`, y en la raíz crear `index.html`.

Paso 3 Escribir `api.js`.

```
1      "use strict";
2
3      const URL_USUARIOS = "https://randomuser.me/api/";
4
5      export async function obtenerUsuarios(cantidad = 12) {
6          const params = new URLSearchParams({ results: cantidad, nat:
7              "es,us" });
8          const res = await fetch(`${URL_USUARIOS}?${params}`);
9          if (!res.ok) throw new Error(`HTTP ${res.status}`);
10         const data = await res.json();
11         return data.results;
12     }
```


Listing 4.24: src/api.js

Qué hace: exporta una función asíncrona que consulta la API y devuelve el array de usuarios. Lanza un error si la respuesta HTTP no es exitosa.

Paso 4 Escribir ui.js.

```
1      "use strict";
2
3      export function crearTarjeta(usuario) {
4          const { name, email, picture, location } = usuario;
5          const tarjeta = document.createElement("article");
6          tarjeta.className = "tarjeta";
7
8          tarjeta.innerHTML = `
9              
10             <h3>${name.first} ${name.last}</h3>
11             <p>${email}</p>
12             <p><small>${location.city}, ${location.country}</small></p>
13             `;
14          return tarjeta;
15      }
16
17      export function mostrarError(msg) {
18          const contenedor = document.getElementById("galeria");
19          contenedor.innerHTML =
20              `<p class="error">Error: ${msg}. Intente de nuevo.</p>`;
21      }
22
23      export function mostrarCargando(activo) {
24          document.getElementById("cargando").hidden = !activo;
25      }
```

Listing 4.25: src/ui.js

Qué hace: encapsula toda la lógica de creación de nodos DOM. No mezcla lógica de red con presentación.

Paso 5 Escribir main.js.

```
1      "use strict";
2      import { obtenerUsuarios } from "../api.js";
3      import { crearTarjeta, mostrarError, mostrarCargando } from
4          "../ui.js";
5
6      const galeria = document.getElementById("galeria");
7
8      async function cargarGaleria() {
9          mostrarCargando(true);
10         galeria.innerHTML = "";
11         try {
12             const usuarios = await obtenerUsuarios(12);
13             const fragmento = document.createDocumentFragment();
```

```

13     usuarios.forEach(u =>
14         fragmento.appendChild(crearTarjeta(u)));
15     galeria.appendChild(fragmento);           // un solo reflow
16 } catch (err) {
17     mostrarError(err.message);
18 } finally {
19     mostrarCargando(false);
20 }
21
22 document.getElementById("btn-recargar")
23 .addEventListener("click", cargarGaleria);
24
25 cargarGaleria();                             // carga inicial

```

Listing 4.26: src/main.js

Por qué DocumentFragment: agregar elementos uno a uno fuerza un *reflow* por cada inserción. Un fragmento agrupa las inserciones y genera un único *reflow*, mejorando el rendimiento.

Paso 6 Escribir index.html.

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
    initial-scale=1">
<title>Galeria de Usuarios</title>
<style>
*, *::before, *::after { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; margin: 0; padding:
    1rem;
    background: #f5f5f5; }
h1 { color: #006633; text-align: center; }
#galeria { display: grid;
    grid-template-columns: repeat(auto-fill, minmax(220px,1fr));
    gap: 1rem; margin-top: 1rem; }
.tarjeta { background: white; border-radius: 8px; padding: 1rem;
    text-align: center; box-shadow: 0 2px 6px rgba(0,0,0,.1); }
.tarjeta img { border-radius: 50%; width: 96px; height: 96px; }
.tarjeta h3 { margin: .5rem 0 .25rem; font-size: 1rem; }
.tarjeta p { margin: 0; color: #555; font-size: .85rem; }
.error { color: red; grid-column: 1/-1; text-align: center; }
button { display: block; margin: 0 auto; padding: .6rem 1.4rem;
    background: #006633; color: white; border: none;
    border-radius: 6px; cursor: pointer; font-size: 1rem; }
button:hover { background: #004d26; }
#cargando { text-align: center; color: #888; margin-top: 2rem; }
</style>
</head>

```

```

<body>
<h1>Galeria de Usuarios</h1>
<button id="btn-recargar">Recargar galeria</button>
<p id="cargando" hidden>Cargando...</p>
<section id="galeria" aria-live="polite"></section>
<script type="module" src="src/main.js"></script>
</body>
</html>

```

Listing 4.27: index.html

Paso 7 Ejecutar y verificar.

En IntelliJ, hacer clic derecho sobre `index.html` y elegir **Open in** → **Chrome** (o el navegador preferido). IntelliJ levanta un servidor HTTP local en `localhost:63342` automáticamente. Abrir las DevTools (F12) → pestaña **Network** y verificar que la petición a `randomuser.me` devuelva 200 y un JSON con 12 usuarios. En la pestaña **Console** no debe aparecer ningún error.

Verificación: la galería debe mostrar 12 tarjetas con foto, nombre, correo y ciudad. Al hacer clic en `Recargar galería` se obtienen 12 usuarios distintos.

4.10 Buenas prácticas de JavaScript 2026

Doce reglas profesionales JavaScript 2026

1. **"use strict"** en todo módulo o `type="module"` en el `<script>` (activa *strict mode* implícitamente).
2. **const por defecto**; `let` solo cuando la variable muta; `var` nunca.
3. **=== sobre ==** siempre (comparación estricta evita coerciones silenciosas).
4. **Manejo de errores** con `try/catch` en todas las funciones `async`; nunca silenciar un `catch` vacío.
5. **Validar entradas** de funciones públicas antes de usarlas; lanzar `TypeError` con mensaje claro.
6. **Funciones pequeñas**, una sola responsabilidad; máximo 20 líneas como heurística.
7. **Sin variables globales**: usar módulos ES con `type="module"`.
8. **Lintor ESLint** con `eslint-config-airbnb` o `eslint-config-standard`; integrarlo en IntelliJ.
9. **Tests** con Vitest o Jest; cobertura mínima 70% en lógica de negocio.
10. **No mezclar** `.then()` y `await` en la misma función: elegir un estilo y ser consistente.
11. **Optional chaining** (`?.`) para acceso seguro a propiedades anidadas; nullish coalescing (`??`) para valores por defecto precisos.
12. **Tipado gradual** con JSDoc (`@param`, `@returns`) o TypeScript en proyectos serios; IntelliJ IDEA proporciona completado de tipos con JSDoc.

Anti-patrones frecuentes que se deben evitar

- **innerHTML con datos del usuario:** riesgo de XSS. Usar `textContent` o `element.insertAdjacentText()` para texto sin etiquetas.
- **Callback hell:** usar `async/await`.
- **Polling** con `setInterval` para actualizar datos: usar `WebSocket` o `Server-Sent Events`.
- **Modificar el DOM en bucle:** usar `DocumentFragment`.
- **Olvidar cancelar peticiones:** usar `AbortController`.
- **Eval():** nunca; riesgo de seguridad e impide optimizaciones del motor V8.

4.11 Diagrama de arquitectura: cliente con módulos ES

La Figura 4.3 muestra la arquitectura típica de una aplicación de cliente moderna organizada con módulos ES.

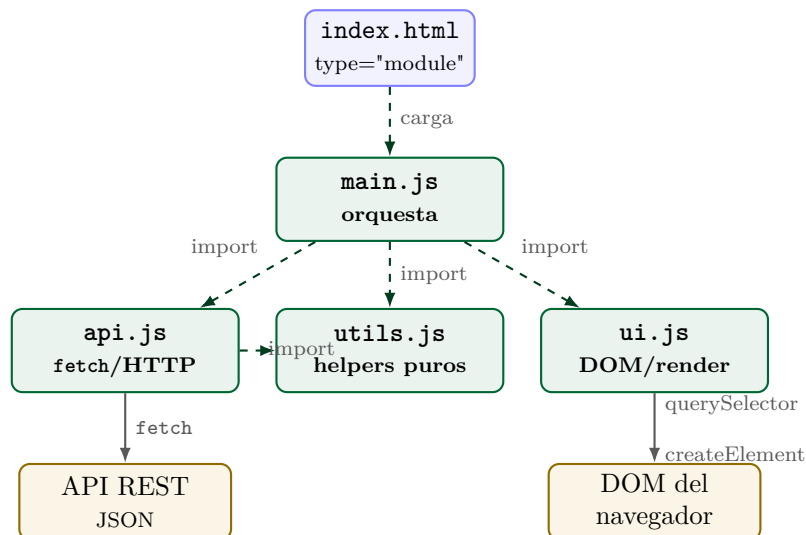


Figura 4.3: Arquitectura de una SPA mínima con módulos ES nativos.

Resumen de la semana

Lo esencial de la semana 4

- JavaScript es **single-threaded** con un *event loop* que permite asincronía sin bloquear la interfaz.
- **const/let**, funciones flecha, template literals, *destructuring* y *spread* son los pilares del JS moderno; **var** está obsoleto.
- Las **funciones de orden superior** (`.map()`, `.filter()`, `.reduce()`) transforman datos de forma funcional e inmutable.
- Los **closures** permiten encapsular estado privado sin necesidad de clases.

- Las **clases ES6** son azúcar sobre prototipos; los campos privados (#) datan de ES2022.
- La **manipulación del DOM** debe ser mínima y usar delegación de eventos para listas dinámicas.
- **async/await** simplifica el código asíncrono; siempre envolver con **try/catch**.
- Los **módulos ES** eliminan el scope global y son el estándar nativo; **AbortController** cancela peticiones.